



Laboratorium : L-204

Waktu : Rabu, 15.45

Minggu Ke : 2

Materi : MySQL Database

Praktikum : Web Services

Jurusan : S1 – INF

Tanggal : 11 Maret 2026

Jenis Soal : Materi & Tugas

MATERI (TOTAL: 35)

Prototipe "Kelontong Bahagia Online" sukses besar, tetapi suatu hari toko Pak Budi mengalami mati lampu. Karena sistem masih menggunakan penyimpanan sementara, begitu program menyala kembali, semua data pelanggan, riwayat saldo, dan sisa stok barang hilang tanpa sisa! Sempat panik luar biasa, Pak Budi akhirnya sadar akan pentingnya keamanan data dan memutuskan untuk merogoh kocek demi beralih ke *database* sungguhan.

Sebagai *Backend Engineer* andalannya, tugas Anda minggu ini adalah melakukan migrasi total. Anda diminta untuk merancang skema *database* relasional menggunakan **MySQL** dan mengubah seluruh operasional *endpoint* mulai dari pencatatan pelanggan, pengelolaan stok barang, hingga transaksi pemotongan saldo agar tersimpan secara permanen dan terstruktur di dalam tabel *database*.

1. Tambah Barang Baru (POST /api/items)

Deskripsi: Endpoint untuk memasukkan data barang baru ke rak toko kelontong.

No	Skenario Test Case	Request	Expected Response & Status Code
1.1	Berhasil Tambah Barang Memasukkan data barang lengkap dan valid.	Method: POST Body (JSON): { "nama_barang": "Kopi Sachet", "kategori": "Minuman", "harga": 3000, "stok": 50 }	Status: 201 Created Response: { "status": "success", "message": "Barang berhasil ditambahkan", "data": { "id_barang": 1, "nama_barang": "Kopi Sachet", "kategori": "Minuman", "harga": 3000, "stok": 50 } }
1.2	Error Validasi: Field Kosong Ada parameter wajib yang tidak dikirim.	Method: POST Body (JSON): { "nama_barang": "Indomie Goreng", "stok": 10 }	Status: 400 Bad Request Response: { "status": "error", "message": "Gagal: kategori dan harga wajib diisi!" }

1.3	Error Validasi: Minus Menginputkan stok atau harga dengan angka di bawah 0.	Method: POST Body (JSON): { "nama_barang": "Sabun Mandi", "kategori": "Sabun", "harga": -500, "stok": 10 }	Status: 400 Bad Request Response: { "status": "error", "message": "Gagal: Harga dan stok tidak boleh minus!" }
-----	---	--	---

2. Lihat Semua Barang (GET /api/items)

Deskripsi: Mengambil daftar barang. Endpoint ini juga harus mendukung filter pencarian berdasarkan kategori menggunakan *Query Params*.

No	Skenario Test Case	Request	Expected Response & Status Code
2.1	Berhasil Menampilkan Semua Memanggil endpoint tanpa filter apa pun.	Method: GET Query Params: (Kosong)	Status: 200 OK Response: { "status": "success", "data": [{ "id_barang": 1, "nama_barang": "Kopi Sachet", "kategori": "Minuman" }] }
2.2	Berhasil Filter Kategori Mencari barang hanya di kategori tertentu.	Method: GET Query Params: ?kategori=Minuman	Status: 200 OK Response: { "status": "success", "data": [{ "id_barang": 1, "nama_barang": "Kopi Sachet", "kategori": "Minuman" }] }
2.3	Error Validasi: Data Kosong Mencari kategori yang tidak ada barangnya di database.	Method: GET Query Params: ?kategori=Elektronik	Status: 404 Not Found Response: { "status": "error", "message": "Barang dengan kategori Elektronik tidak ditemukan" }

3. Lihat Detail 1 Barang (GET /api/items/:id)

Deskripsi: Menampilkan detail barang secara spesifik menggunakan parameter ID di URL (*Path Parameter*).

No	Skenario Test Case	Request	Expected Response & Status Code
3.1	Berhasil Lihat Detail	Method: GET URL: /api/items/1	Status: 200 OK

	Mencari barang dengan ID yang ada di DB.		Response: <pre>{ "status": "success", "data": { "id_barang": 1, "nama_barang": "Kopi Sachet", "stok": 50 } }</pre>
3.2	Error Validasi: ID Tidak Ada Mencari ID yang belum pernah didaftarkan.	Method: GET URL: /api/items/99	Status: 404 Not Found Response: <pre>{ "status": "error", "message": "Barang dengan ID 99 tidak ditemukan di database" }</pre>

4. Update Stok atau Harga Barang (PUT /api/items/:id)

Deskripsi: Memperbarui data stok atau harga dari barang yang sudah ada. Validasi minus dan validasi ID tetap berlaku.

No	Skenario Test Case	Request	Expected Response & Status Code
4.1	Berhasil Update Data Mengubah stok Kopi dari 50 menjadi 45 karena laku terjual.	Method: PUT URL: /api/items/1 Body (JSON): <pre>{ "stok": 45 }</pre>	Status: 200 OK Response: <pre>{ "status": "success", "message": "Data barang berhasil diupdate", "data": { "id_barang": 1, "stok": 45 } }</pre>
4.2	Berhasil Update Data Mengubah stok Kopi dari 50 menjadi 45 karena laku terjual.	Method: PUT URL: /api/items/1 Body (JSON): <pre>{ "harga": 10000 }</pre>	Status: 200 OK Response: <pre>{ "status": "success", "message": "Data barang berhasil diupdate", "data": { "id_barang": 1, "harga": 10000 } }</pre>
4.3	Error Validasi: ID Tidak Ditemukan	Method: PUT URL: /api/items/99	Status: 404 Not Found Response:

Mencoba mengupdate ID yang tidak valid.	Body (JSON): <pre>{ "stok": 10 }</pre>	<pre>{ "status": "error", "message": "Gagal update. Barang dengan ID 99 tidak ada." }</pre>
---	--	---

5. Hapus Barang dari Toko (DELETE /api/items/:id)

Deskripsi: Menghapus barang secara permanen dari database (*Hard Delete*) karena barang sudah tidak dijual lagi.

s	Skenario Test Case	Request	Expected Response & Status Code
5.1	Berhasil Menghapus Permanen Menghapus ID barang yang valid di database.	Method: DELETE URL: /api/items/1	Status: 200 OK Response: <pre>{ "status": "success", "message": "Barang Kopi Sachet berhasil dihapus dari sistem" }</pre>
5.2	Error Validasi: Hapus ID Kosong Mencoba menghapus data yang tidak ada di DB.	Method: DELETE URL: /api/items/99	Status: 404 Not Found Response: <pre>{ "status": "error", "message": "Gagal menghapus. ID Barang 99 tidak ditemukan." }</pre>

**DILARANG MENGGUNAKAN MATERI YANG BELUM DIAJARKAN PADA MINGGU INI
JIKA MELANGGAR MAKA NILAI MATERI : 0**

PERHATIKAN KETENTUAN DIBAWAH :

- Highlight kriteria yang dikerjakan dengan warna kuning dan kumpulkan word beserta dengan file materi, apabila tidak dikumpulkan maka materi tidak akan diperiksa.
- Akan ada pengurangan nilai sebesar -5 untuk setiap kriteria yang dihighlight namun tidak dikerjakan.
- MENCONTEK = Nilai MOD 2

MATERI : 35

SCORE	KRITERIA
0/7	Endpoint POST barang berjalan dengan benar beserta validasi
0/5	Endpoint GET barang berjalan dengan benar beserta filter dan validasi
0/5	Endpoint GET detail barang berjalan dengan benar beserta validasi
0/6	Endpoint PUT barang berjalan dengan benar beserta validasi
0/4	Endpoint DELETE barang berjalan dengan benar beserta validasi
0/4	Semua HTTP status code benar
0/4	Format response mengikuti standar (status, message, data)
Total : 35	

TUGAS (TOTAL = 25)

MURR Badminton Court adalah salah satu GOR Badminton teramai di kota. Selama ini, Pak Budi selaku pemilik GOR masih menggunakan buku catatan manual untuk mencatat jadwal sewa lapangan. Hal ini sering menimbulkan masalah yang fatal: *double-booking* (lapangan yang sama dipesan dua orang di jam yang sama), kesulitan menghitung pendapatan karena harga sewa yang bervariasi setiap waktu, hingga keluhan pelanggan.

Selain itu, Pak Budi juga sering menerima pembatalan jadwal dari pelanggan. Namun, ia **tidak mau data pembatalan itu hilang dari sistem** agar ia tetap bisa melacak riwayat pemesanan pelanggan yang suka membatalkan secara sepihak (*Soft Delete*). Pak Budi juga sering kali kebingungan mencari data jika pelanggan datang dan hanya menyebutkan namanya. Tugas Anda adalah merancang REST API yang kompleks dan *solid* untuk mengotomatisasi seluruh sistem MURR Badminton Court.

1. **Fasilitas GOR:** Terdapat **8 Lapangan** (Nomor 1 - 8).
2. **Aturan Harga (Per Jam):**
 - **Pagi (06:00 - 11:59):** Rp 55.000 / jam
 - **Siang (12:00 - 17:59):** Rp 65.000 / jam
 - **Malam (18:00 - 23:59):** Rp 75.000 / jam
3. **Pengecekan Bentrok (Conflict):** Lapangan yang sudah dipesan tidak boleh dipesan lagi di jam yang sama, **KECUALI** status pesanan tersebut sudah dibatalkan (*Soft Deleted*).
4. **Logika Soft Delete:** Saat data dihapus (DELETE), data **tidak boleh terhapus dari Database**. Anda wajib menggunakan field seperti `is_deleted: true` atau `status: "cancelled"`.
5. Semua endpoint GET (kecuali history pembatalan) **hanya boleh** menampilkan data yang belum di-soft delete (`is_deleted: false`).

Berikut adalah endpoint yang harus dibuat:

1. Registrasi Pelanggan Baru (POST /api/customers)

Aturan: Menyimpan data pelanggan baru sebelum mereka bisa melakukan reservasi. Validasi duplikasi nomor WhatsApp.

No	Skenario Test Case	Request	Expected Response & Status Code
1.1	Berhasil Register Memasukkan data pelanggan dengan lengkap dan benar.	Method: POST Body (JSON): <pre>{ "nama_pelanggan": "Andi Kusuma", "no_whatsapp": "08123456789" }</pre>	Status: 201 Created Response: <pre>{ "status": "success", "message": "Pelanggan berhasil didaftarkan", "data": { "id_pelanggan": 1, "nama_pelanggan": "Andi Kusuma" } }</pre>

1.2	Error Validasi: Field Kosong User tidak mengirimkan field yang wajib diisi.	Method: POST Body (JSON): <pre>{ "nama_pelanggan": "Budi" }</pre>	Status: 400 Bad Request Response: <pre>{ "status": "error", "message": "Validasi gagal: no_whatsapp wajib diisi!" }</pre>
1.3	Error Validasi: WA Duplikat Mendaftar dengan nomor WA yang sudah ada di database.	Method: POST Body (JSON): <pre>{ "nama_pelanggan": "Citra", "no_whatsapp": "08123456789" }</pre>	Status: 409 Conflict Response: <pre>{ "status": "error", "message": "Nomor WhatsApp sudah terdaftar di sistem" }</pre>

2. Buat Reservasi Baru (POST /api/bookings)

Aturan: Wajib validasi kelengkapan input. Jika nomor_lapangan tidak diisi, sistem harus *auto-generate* mencari lapangan 1-8 yang kosong di jadwal tersebut. Harga dihitung otomatis berdasarkan durasi dan *shift* jam.

No	Skenario Test Case	Request	Expected Response & Status Code
2.1	Berhasil: Lapangan Spesifik User memilih lapangan spesifik (id_lapangan: 3) dan tidak bentrok.	Method: POST Body (JSON): <pre>{ "id_pelanggan": 1, "tanggal": "2026-03-20", "jam_mulai": "19:00", "durasi": 2, "id_lapangan": 3 }</pre>	Status: 201 Created Response: <pre>{ "status": "success", "message": "Reservasi berhasil", "data": { "id_reservasi": 1, "id_lapangan": 3, "total_harga": 150000 } }</pre>
2.2	Berhasil: Auto-Generate Lapangan User tidak mengirim id_lapangan. Sistem mencari otomatis.	Method: POST Body (JSON): <pre>{ "id_pelanggan": 1, "tanggal": "2026-03-20", "jam_mulai": "10:00", "durasi": 1 }</pre>	Status: 201 Created Response: <pre>{ "status": "success", "message": "Lapangan otomatis dipilihkan", "data": { "id_reservasi": 2, "id_lapangan": 1, "total_harga": 55000 } }</pre>
2.3	Error Validasi: Field Kosong Atribut wajib untuk reservasi tidak lengkap.	Method: POST Body (JSON): <pre>{ "id_pelanggan": 1,</pre>	Status: 400 Bad Request Response: <pre>{ "status": "error",</pre>

		"durasi": 2 }	"message": "Validasi gagal: tanggal dan jam_mulai wajib diisi" }
2.4	Error Validasi: Pelanggan Invalid Mengirim id_pelanggan yang tidak ada di database tabel pelanggan.	Method: POST Body (JSON): { "id_pelanggan": 999, "tanggal": "2026-03-20", "jam_mulai": "10:00", "durasi": 1 }	Status: 404 Not Found Response: { "status": "error", "message": "Pelanggan dengan ID 999 tidak terdaftar" }
2.5	Error Validasi: Bentrok (Conflict) Memilih jadwal di id_lapangan yang sudah berstatus dipesan.	Method: POST Body (JSON): { "id_pelanggan": 1, "tanggal": "2026-03-20", "jam_mulai": "19:00", "durasi": 1, "id_lapangan": 3 }	Status: 409 Conflict Response: { "status": "error", "message": "Maaf, Lapangan tersebut sudah dipesan di jam ini" }

3. Lihat Semua Reservasi Aktif (GET /api/bookings)

Aturan: Hanya menampilkan data dengan is_deleted: false. Mendukung pencarian via *Query Params*.

No	Skenario Test Case	Request	Expected Response & Status Code
3.1	Berhasil: Filter dengan JOIN Mendapatkan jadwal berdasarkan parameter tanggal.	Method: GET Query Params: ?tanggal=2026-03-20	Status: 200 OK Response: { "status": "success", "data": [{ "id_reservasi": 1, "nama_pelanggan": "Andi Kusuma", "nomor_lapangan": 3, "jam_mulai": "19:00", "total_harga": 150000 }] }
3.2	Error Validasi: Data Kosong Mencari di tanggal yang tidak memiliki satupun jadwal.	Method: GET Query Params: ?tanggal=2026-12-12	Status: 404 Not Found Response: { "status": "error", "message": "Tidak ada data reservasi pada tanggal tersebut" }

4. Cari Reservasi by Nama Pemesan (GET /api/bookings/customer/:nama)

Aturan: Pencarian nama bersifat *case-insensitive* (tidak peka huruf besar/kecil) dan menggunakan fitur *LIKE* / *Regex* (pencarian sebagian kata). Hanya data aktif.

No	Skenario Test Case	Request	Expected Response & Status Code
4.1	Berhasil Menemukan Relasi Mencari potongan nama pelanggan, menampilkan semua pesannya.	Method: GET Path Params: nama = andi	Status: 200 OK Response: <pre>{ "status": "success", "data": { "id_pelanggan": 1, "nama_pelanggan": "Andi Kusuma", "riwayat_reservasi": [{ "id_reservasi": 1, "tanggal": "2026-03-20" }] } }</pre>
4.2	Error Validasi: Nama Tidak Ada Mencari nama yang tidak terdaftar sama sekali di database.	Method: GET Path Params: nama = xavier	Status: 404 Not Found Response: <pre>{ "status": "error", "message": "Data pelanggan xavier tidak ditemukan" }</pre>

5. Lihat Detail & Ubah Jadwal (GET & PUT /api/bookings/:id)

Aturan GET: Menampilkan detail 1 data reservasi secara spesifik berdasarkan ID. Wajib melakukan JOIN agar nama pelanggan dan nomor lapangan muncul.

Aturan PUT: Jika memindah jam, sistem wajib menghitung ulang harga. Lapangan baru juga harus dicek ketersediaannya (pastikan tidak bentrok).

No	Skenario Test Case	Request	Expected Response & Status Code
5.1	Berhasil Lihat Detail (GET) Menampilkan satu data reservasi spesifik beserta relasi nama pelanggan.	Method: GET URL: /api/bookings/1	Status: 200 OK Response: <pre>{ "status": "success", "data": { "id_reservasi": 1, "nama_pelanggan": "Andi Kusuma", "nomor_lapangan": 3, "jam_mulai": "19:00", "total_harga": 150000 } }</pre>
5.2	Error Validasi: ID Detail Tidak Ada (GET)	Method: GET URL: /api/bookings/999	Status: 404 Not Found Response:

	Mencari ID reservasi yang tidak ada di database.		{ "status": "error", "message": "Data reservasi dengan ID 999 tidak ditemukan" }
5.3	Berhasil: Update & Hitung Ulang (PUT) Pindah dari shift Malam (75k) ke Pagi (55k). Harga otomatis berubah.	Method: PUT URL: /api/bookings/1 Body (JSON): { "jam_mulai": "09:00" }	Status: 200 OK Response: { "status": "success", "message": "Jadwal diupdate", "data": { "id_reservasi": 1, "jam_mulai": "09:00", "total_harga": 110000 } }
5.4	Error Validasi: Pindah ke Slot Bentrok (PUT) Mengubah lapangan ke lapangan yang sudah diboooking orang lain.	Method: PUT URL: /api/bookings/1 Body (JSON): { "id_lapangan": 2 }	Status: 409 Conflict Response: { "status": "error", "message": "Gagal update. Lapangan tersebut penuh di jam ini" }
5.5	Error Validasi: ID Update Tidak Ditemukan (PUT) Mengirimkan parameter ID ke database yang kosong saat akan update.	Method: PUT URL: /api/bookings/999 Body (JSON): { "jam_mulai": "09:00" }	Status: 404 Not Found Response: { "status": "error", "message": "ID Reservasi 999 tidak valid untuk diupdate" }

6. Update Status Pembayaran (PUT /api/bookings/:id/payment)

Aturan: Endpoint ini hanya memperbarui status bayar. Menerima body status_pembayaran. Lakukan validasi input.

No	Skenario Test Case	Request	Expected Response & Status Code
6.1	Berhasil Ubah Status Melakukan update status bayar ke lunas.	Method: PUT URL: /api/bookings/1/payment Body (JSON): { "status_pembayaran": "lunas" }	Status: 200 OK Response: { "status": "success", "message": "Pembayaran berhasil diupdate" }

6.2	Error Validasi: Status Tidak Sesuai User menginputkan kata yang tidak terdaftar di sistem.	Method: PUT URL: /api/bookings/1/payment Body (JSON): <pre>{ "status_pembayaran": "ngutang" }</pre>	Status: 400 Bad Request Response: <pre>{ "status": "error", "message": "Status hanya boleh: belum_bayar atau lunas" }</pre>
-----	--	---	---

7. Pembatalan Reservasi (Soft DELETE /api/bookings/:id)

Aturan: Harus menggunakan **SOFT DELETE**.

No	Skenario Test Case	Request	Expected Response & Status Code
7.1	Berhasil Hapus (Soft Delete) Membatalkan pesanan yang valid.	Method: DELETE URL: /api/bookings/1	Status: 200 OK Response: <pre>{ "status": "success", "message": "Reservasi berhasil dibatalkan (Soft Delete)" }</pre>
7.2	Error Validasi: Data Sudah Dihapus Mencoba menghapus data yang statusnya sudah terhapus.	Method: DELETE URL: /api/bookings/1	Status: 400 Bad Request Response: <pre>{ "status": "error", "message": "Reservasi ini sudah dibatalkan sebelumnya" }</pre>

8. Lihat Riwayat Pembatalan (GET /api/bookings/history/trash)

Aturan: Endpoint khusus ini hanya menampilkan data yang sudah di-*Soft Delete* (is_deleted: true).

No	Skenario Test Case	Request	Expected Response & Status Code
8.1	Berhasil Menampilkan Data Terhapus Memanggil endpoint trash history.	Method: GET URL: /api/bookings/trash/history	Status: 200 OK Response: <pre>{ "status": "success", "data": [{ "id_reservasi": 1, "nama_pelanggan": "Andi Kusuma", "is_deleted": true }] }</pre>

**DILARANG MENGGUNAKAN KONSEP MAUPUN MATERI YANG BELUM DIAJARKAN
JIKA MELANGGAR MAKA NILAI TUGAS: 0**

WAJIB HAPUS NODE MODULES ATAU NILAI TUGAS: 0

FORMAT DATABASE: T2_<9 DIGIT NRP>.sql

PERHATIKAN KETENTUAN DIBAWAH :

- Highlight kriteria yang dikerjakan dengan warna kuning dan kumpulkan word beserta dengan file tugas, apabila tidak dikumpulkan maka tugas tidak akan diperiksa.
- Akan ada pengurangan nilai sebesar -5 untuk setiap kriteria yang dihighlight namun tidak dikerjakan.
- **MENCONTEK = Nilai MOD 2**

TUGAS : 25

SCORE	KRITERIA
0/3	Endpoint POST customer berjalan dengan benar beserta validasi
0/3	Endpoint POST booking berjalan dengan benar beserta validasi
0/2	Endpoint GET booking berjalan dengan benar beserta validasi
0/2	Endpoint GET booking by nama barang berjalan dengan benar beserta validasi
0/2	Endpoint GET booking by id berjalan dengan benar beserta validasi
0/3	Endpoint PUT booking berjalan dengan benar beserta validasi
0/3	Endpoint PUT status pembayaran berjalan dengan benar beserta validasi
0/2	Endpoint DELETE booking berjalan dengan benar beserta validasi (DICEK JIKA SOFT DELETE)
0/2	Endpoint GET riwayat booking yang sudah dibatalkan berjalan dengan benar beserta validasi
0/1	Semua HTTP status code benar
0/2	Format response mengikuti standar (status, message, data)
Total : 25	

Menyetujui

Mengetahui

Penyusun Soal

(Mikhael Setiawan, S.Kom.,
M.Kom.)

Koordinator Kuliah

(Grace Levina Dewi, M.Kom.)
Koordinator Laboratorium

(Vincentius Jason Tjendika)
Asisten